Understanding credit risk

CREDIT RISK MODELING IN PYTHON

Michael Crabtree
Data Scientist, Ford Motor Company

What is credit risk?

- The possibility that someone who has borrowed money will not repay it all
- Calculated risk difference between lending someone money and a government bond
- When someone fails to repay a loan, it is said to be in default
- The likelihood that someone will default on a loan is the probability of default (PD)

What is credit risk?

- The possibility that someone who has borrowed money will not repay it all
- Calculated risk difference between lending someone money and a government bond
- When someone fails to repay a loan, it is said to be in default
- The likelihood that someone will default on a loan is the probability of default (PD)

Payment	Payment Date	Loan Status
$100	Jun 15	Non-Default
$100	Jul 15	Non-Default
$0	Aug 15	Default

Expected loss

- The dollar amount the firm loses as a result of loan default
- Three primary components:
  - Probability of Default (PD)
  - Exposure at Default (EAD)
  - Loss Given Default (LGD)

Formula for expected loss:

```
expected_loss = PD * EAD * LGD
```

Types of data used

Two Primary types of data used:

- Application data
- Behavioral data

Application	Behavioral
Interest Rate	Employment Length
Grade	Historical Default
Amount	Income

Data columns

- Mix of behavioral and application
- Contain columns simulating credit bureau data

Column	Column
Income	Loan grade
Age	Loan amount
Home ownership	Interest rate
Employment length	Loan status
Loan intent	Historical default
Percent Income	Credit history length

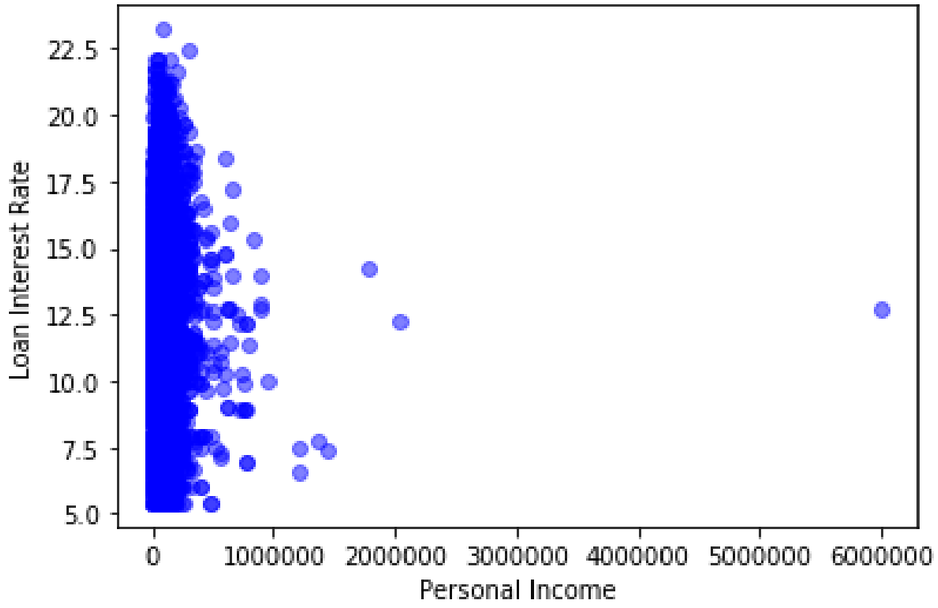
Exploring with cross tables

```
pd.crosstab(cr_loan['person_home_ownership'], cr_loan['loan_status'],
            values=cr_loan['loan_int_rate'], aggfunc='mean').round(2)
```

loan_status	0	1
person_home_ownership		
MORTGAGE	10.06	13.43
OTHER	11.53	13.77
OWN	10.75	12.24
RENT	10.78	13.73

Exploring with visuals

```
plt.scatter(cr_loan['person_income'], cr_loan['loan_int_rate'],c='blue', alpha=0.5)
plt.xlabel("Personal Income")
plt.ylabel("Loan Interest Rate")
plt.show()
```



Let's practice!

CREDIT RISK MODELING IN PYTHON

Outliers in Credit Data

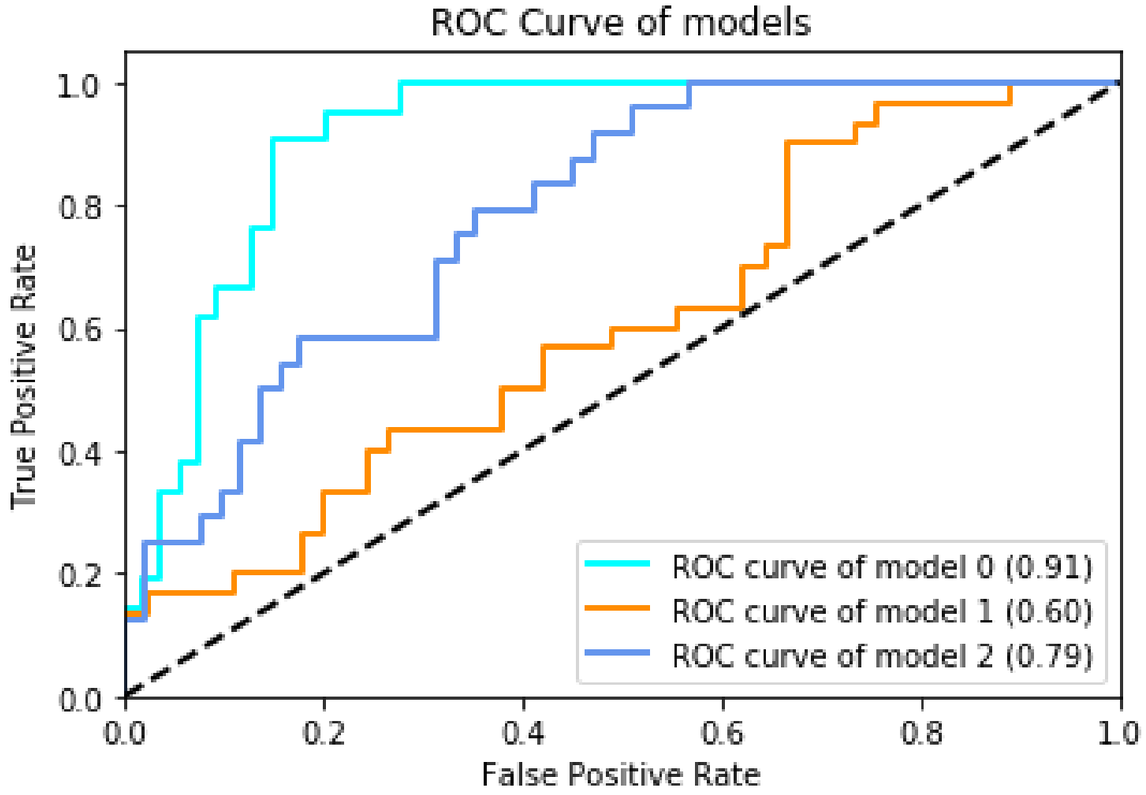
CREDIT RISK MODELING IN PYTHON

Michael Crabtree
Data Scientist, Ford Motor Company

datacamp

Data processing

- Prepared data allows models to train faster
- Often positively impacts model performance



Outliers and performance

Possible causes of outliers:

- Problems with data entry systems (human error)
- Issues with data ingestion tools

Outliers and performance

Possible causes of outliers:

- Problems with data entry systems (human error)
- Issues with data ingestion tools

Feature	Coefficient With Outliers	Coefficient Without Outliers
Interest Rate	0.2	0.01
Employment Length	0.5	0.6
Income	0.6	0.75

Detecting outliers with cross tables

- Use cross tables with aggregate functions

```
pd.crosstab(cr_loan['person_home_ownership'], cr_loan['loan_status'],
            values=cr_loan['loan_int_rate'], aggfunc='mean').round(2)
```

Without Outliers

loan_status	0	1
person_home_ownership		
MORTGAGE	10.06	13.43
OTHER	11.53	13.77
OWN	10.75	12.24
RENT	10.78	13.73

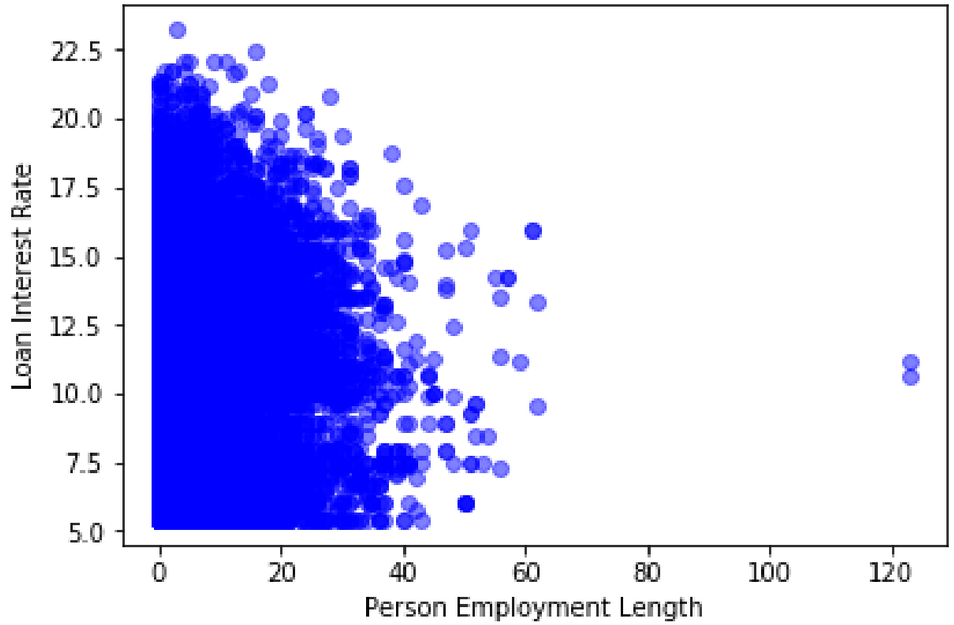
With Outliers

loan_status	0	1
person_home_ownership		
MORTGAGE	10.06	13.43
OTHER	11.53	13.77
OWN	10.75	59183.79
RENT	10.78	13.73

Detecting outliers visually

Detecting outliers visually

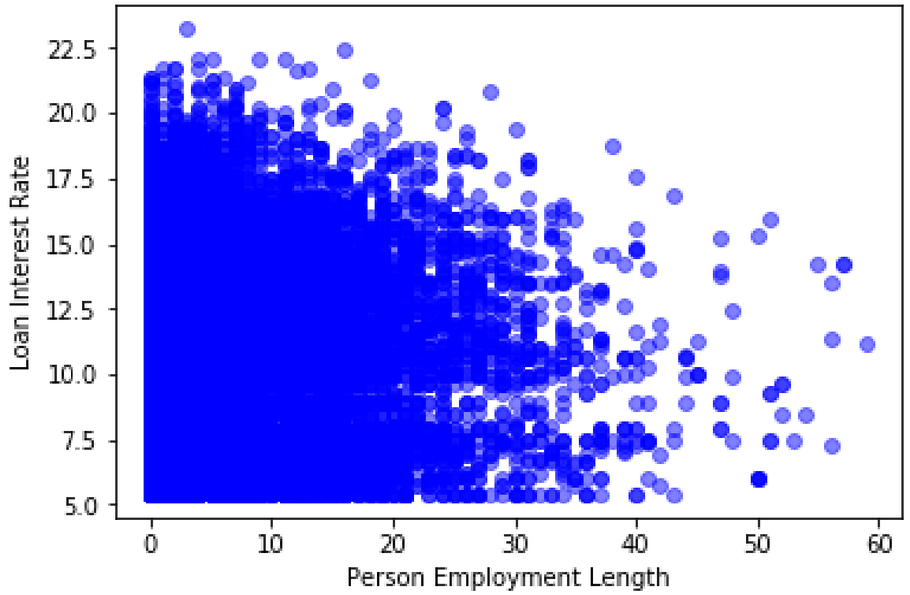
- Histograms
- Scatter plots



Removing outliers

- Use the `.drop()` method within Pandas

```
indices = cr_loan[cr_loan['person_emp_length'] >= 60].index
cr_loan.drop(indices, inplace=True)
```



Let's practice!

CREDIT RISK MODELING IN PYTHON

Risk with missing data in loan data

CREDIT RISK MODELING IN PYTHON

Michael Crabtree
Data Scientist, Ford Motor Company

datacamp

What is missing data?

- NULLs in a row instead of an actual value
- An empty string `''`
- Not an entirely empty row
- Can occur in any column in the data

	person_age	person_income	person_home_ownership	person_emp_length	loan_intent
105	22	12600.0	MORTGAGE	NaN	PERSONAL
222	24	185000.0	MORTGAGE	NaN	EDUCATION
379	24	16800.0	MORTGAGE	NaN	DEBTCONSOLIDATION

Similarities with outliers

- Negatively affect machine learning model performance
- May bias models in unanticipated ways
- May cause errors for some machine learning models

Similarities with outliers

- Negatively affect machine learning model performance
- May bias models in unanticipated ways
- May cause errors for some machine learning models

Missing Data Type	Possible Result
NULL in numeric column	Error
NULL in string column	Error

How to handle missing data

- Generally three ways to handle missing data
  - Replace values where the data is missing
  - Remove the rows containing missing data
  - Leave the rows with missing data unchanged
- Understanding the data determines the course of action

How to handle missing data

- Generally three ways to handle missing data
  - Replace values where the data is missing
  - Remove the rows containing missing data
  - Leave the rows with missing data unchanged
- Understanding the data determines the course of action

Missing Data	Interpretation	Action
NULL in `loan_status`	Loan recently approved	Remove from prediction data
NULL in `person_age`	Age not recorded or disclosed	Replace with median

Finding missing data

- Null values are easily found by using the `isnull()` function
- Null records can easily be counted with the `sum()` function
- `.any()` method checks all columns

```
null_columns = cr_loan.columns[cr_loan.isnull().any()]
cr_loan[null_columns].isnull().sum()
```

```
# Total number of null values per column
person_home_ownership          25
person_emp_length             895
loan_intent                    25
loan_int_rate                3140
cb_person_default_on_file      15
```

Replacing Missing data

- Replace the missing data using methods like `.fillna()` with aggregate functions and methods

```
cr_loan['loan_int_rate'].fillna((cr_loan['loan_int_rate'].mean()), inplace = True)
```

loan_int_rate
5.42
12.42
NaN
10.74
15.27

loan_int_rate
5.420000
12.420000
11.010729
10.740000
15.270000

Dropping missing data

- Uses indices to identify records the same as with outliers
- Remove the records entirely using the `.drop()` method

```
indices = cr_loan[cr_loan['person_emp_length'].isnull()].index
cr_loan.drop(indices, inplace=True)
```

Let's practice!

CREDIT RISK MODELING IN PYTHON

Logistic regression for probability of default

CREDIT RISK MODELING IN PYTHON

Michael Crabtree
Data Scientist, Ford Motor Company

datacamp

Probability of default

- The likelihood that someone will default on a loan is the probability of default
- A probability value between 0 and 1 like `0.86`
- `loan_status` of `1` is a default or `0` for non-default

Probability of default

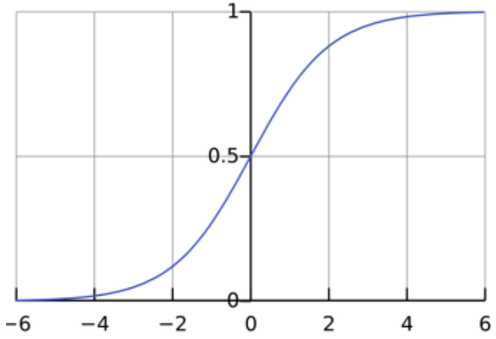
- The likelihood that someone will default on a loan is the probability of default
- A probability value between 0 and 1 like `0.86`
- `loan_status` of `1` is a default or `0` for non-default

Probability of Default	Interpretation	Predicted loan status
0.4	Unlikely to default	0
0.90	Very likely to default	1
0.1	Very unlikely to default	0

Predicting probabilities

- Probabilities of default as an outcome from machine learning
  - Learn from data in columns (features)
- Classification models (default, non-default)
- Two most common models:
  - Logistic regression
  - Decision tree

Logistic Regression



Decision Tree

Logistic regression

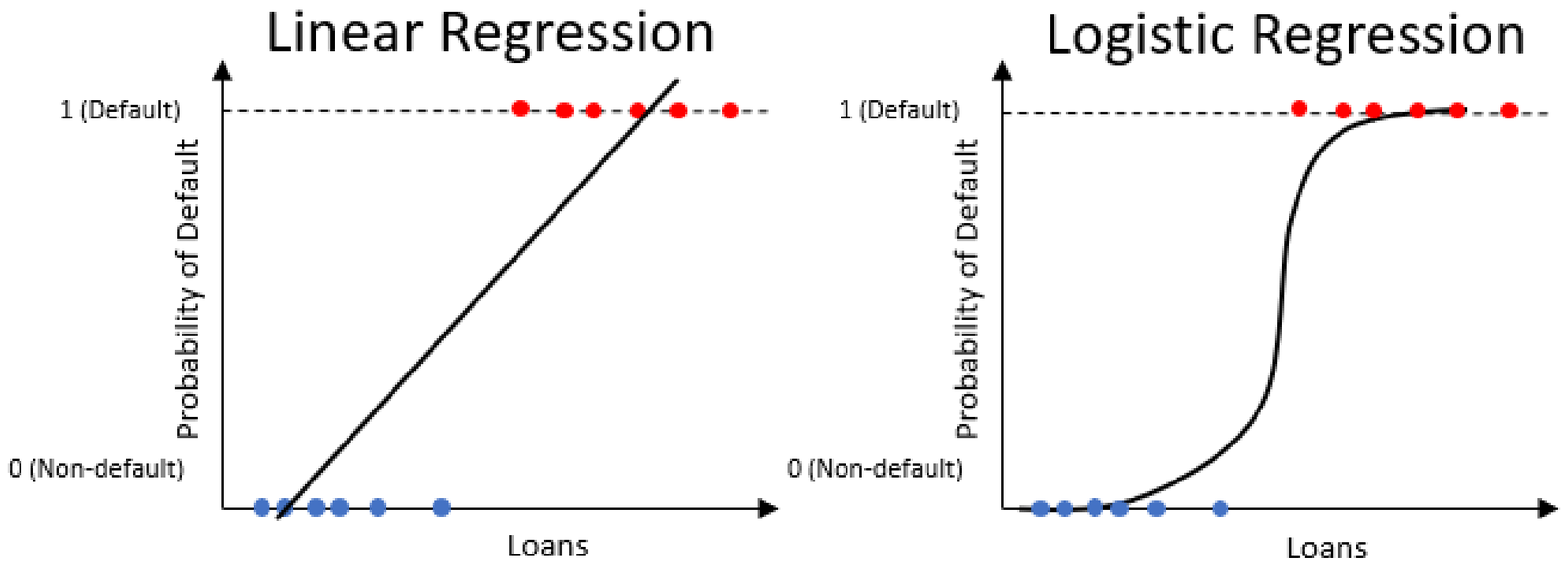
- Similar to the linear regression, but only produces values between `0` and `1`

Linear Regression

$$Y = \beta_0 + (\beta_1 * X_1) + (\beta_2 * X_2) \ldots$$

Logistic Regression

$$P(loan\_status = 1) = \frac{1}{1 + e^{-Y}}$$



Training a logistic regression

- Logistic regression available within the scikit-learn package

```
from sklearn.linear_model import LogisticRegression
```

- Called as a function with or without parameters

```
clf_logistic = LogisticRegression(solver='lbfgs')
```

- Uses the method `.fit()` to train

```
clf_logistic.fit(training_columns, np.ravel(training_labels))
```

- Training Columns: all of the columns in our data **except** `loan_status`
- Labels: `loan_status` (0,1)

Training and testing

- Entire data set is usually split into two parts

Training and testing

- Entire data set is usually split into two parts

Data Subset	Usage	Portion
Train	Learn from the data to generate predictions	60%
Test	Test learning on new unseen data	40%

Creating the training and test sets

- Separate the data into training columns and labels

```
X = cr_loan.drop('loan_status', axis = 1)
y = cr_loan[['loan_status']]
```

- Use `train_test_split()` function already within sci-kit learn

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=.4, random_state=123)
```

- `test_size` : percentage of data for test set
- `random_state` : a random seed value for reproducibility

Let's practice!

CREDIT RISK MODELING IN PYTHON

Predicting the probability of default

CREDIT RISK MODELING IN PYTHON

Michael Crabtree
Data Scientist, Ford Motor Company

datacamp

Logistic regression coefficients

```
# Model Intercept
array([-3.30582292e-10])
# Coefficients for ['loan_int_rate','person_emp_length','person_income']
array([[ 1.28517496e-09, -2.27622202e-09, -2.17211991e-05]])
```

$$P(loan\_status = 1) = \frac{1}{1 + e^{-\left(-3.3e^{-10} + (1.29e^{-9} * Int\ Rate) + (-2.28e^{-9} * Emp\ Length) + (-2.17e^{-5} * Income)\right)}}$$

```
# Calculating probability of default
int_coef_sum = -3.3e-10 +
    (1.29e-09 * loan_int_rate) + (-2.28e-09 * person_emp_length) + (-2.17e-05 * person_income)
prob_default = 1 / (1 + np.exp(-int_coef_sum))
prob_nondefault = 1 - (1 / (1 + np.exp(-int_coef_sum)))
```

Interpreting coefficients

```
# Intercept
intercept = -1.02
# Coefficient for employment length
person_emp_length_coef = -0.056
```

- For every 1 year increase in `person_emp_length` , the person is less likely to default

Interpreting coefficients

```
# Intercept
intercept = -1.02
# Coefficient for employment length
person_emp_length_coef = -0.056
```

- For every 1 year increase in `person_emp_length` , the person is less likely to default

intercept	person_emp_length	value * coef	probability of default
`-1.02`	10	(10 * `-0.06`)	.17
`-1.02`	11	(11 * `-0.06`)	.16
`-1.02`	12	(12 * `-0.06`)	.15

Using non-numeric columns

- Numeric: `loan_int_rate`, `person_emp_length`, `person_income`
- Non-numeric:

```
cr_loan_clean['loan_intent']
```

```
EDUCATION
MEDICAL
VENTURE
PERSONAL
DEBTCONSOLIDATION
HOMEIMPROVEMENT
```

- Will cause errors with machine learning models in Python unless processed

One-hot encoding

- Represent a string with a number

	person_age	person_income	loan_intent
0	21	9600	EDUCATION
1	25	9600	MEDICAL
2	23	65500	MEDICAL
3	24	54400	MEDICAL
4	21	9900	VENTURE

One-hot encoding

- Represent a string with a number
- `0` or `1` in a new column `column_VALUE`

	person_age	person_income	loan_intent
0	21	9600	EDUCATION
1	25	9600	MEDICAL
2	23	65500	MEDICAL
3	24	54400	MEDICAL
4	21	9900	VENTURE

	person_age	person_income	loan_intent_EDUCATION	loan_intent_MEDICAL	loan_intent_VENTURE
0	21	9600	1	0	0
1	25	9600	0	1	0
2	23	65500	0	1	0
3	24	54400	0	1	0
4	21	9900	0	0	1

Get dummies

- Utilize the `get_dummies()` within `pandas`

```
# Separate the numeric columns
cred_num = cr_loan.select_dtypes(exclude=['object'])
# Separate non-numeric columns
cred_cat = cr_loan.select_dtypes(include=['object'])
# One-hot encode the non-numeric columns only
cred_cat_onehot = pd.get_dummies(cred_cat)
# Union the numeric columns with the one-hot encoded columns
cr_loan = pd.concat([cred_num, cred_cat_onehot], axis=1)
```

Predicting the future, probably

- Use the `.predict_proba()` method within scikit-learn

```
# Train the model
clf_logistic.fit(X_train, np.ravel(y_train))
# Predict using the model
clf_logistic.predict_proba(X_test)
```

- Creates array of probabilities of default

```
# Probabilities: [[non-default, default]]
array([[0.55, 0.45]])
```

Let's practice!

CREDIT RISK MODELING IN PYTHON

Credit model performance

CREDIT RISK MODELING IN PYTHON

Michael Crabtree
Data Scientist, Ford Motor Company

datacamp

Model accuracy scoring

- Calculate accuracy

$$Accuracy = \frac{Number\ of\ correct\ predictions}{Number\ of\ predictions}$$

- Use the `.score()` method from scikit-learn

```
# Check the accuracy against the test data
clf_logistic1.score(X_test,y_test)
```

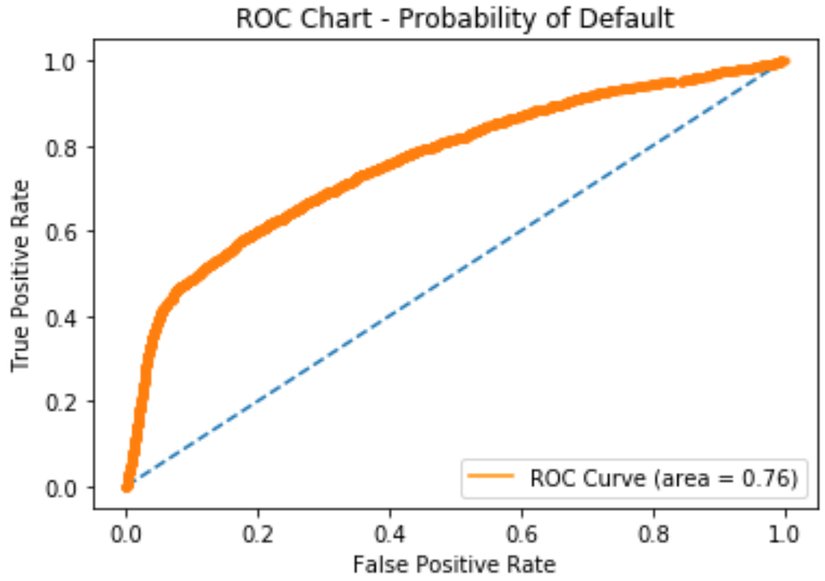
```
0.81
```

- 81% of values for `loan_status` predicted correctly

ROC curve charts

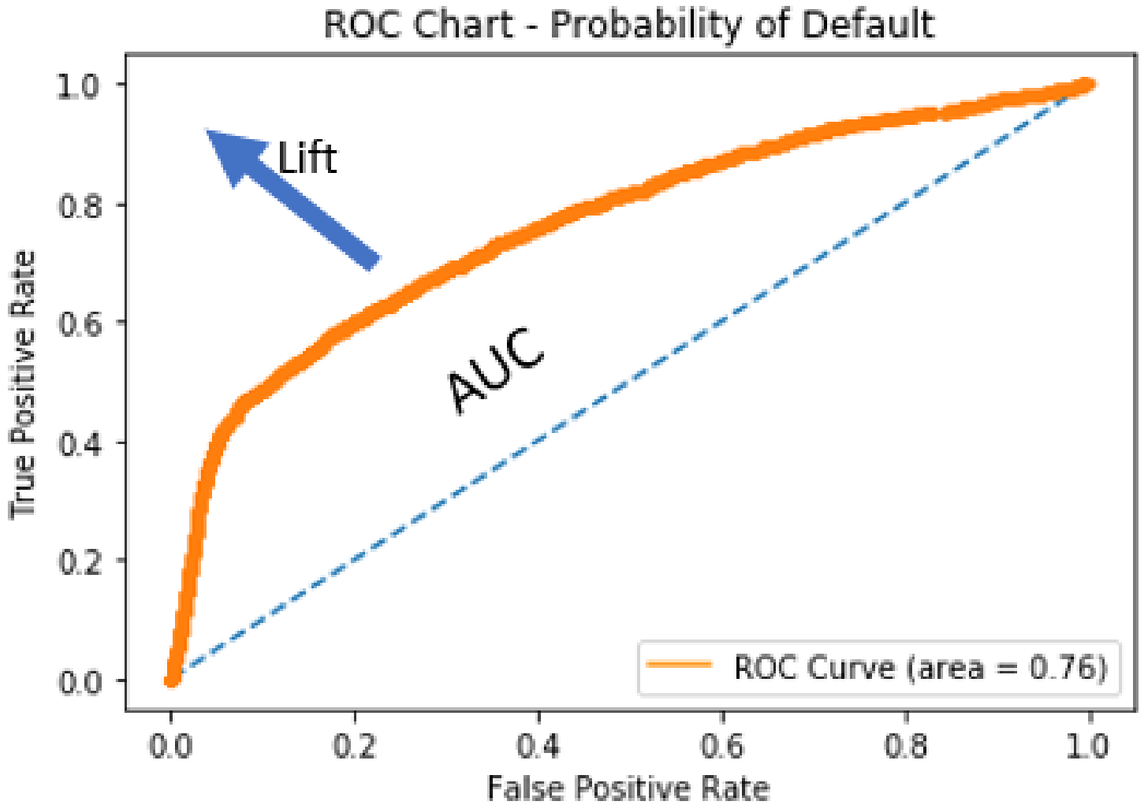
- Receiver Operating Characteristic curve
  - Plots true positive rate (sensitivity) against false positive rate (fall-out)

```
fallout, sensitivity, thresholds = roc_curve(y_test, prob_default)
plt.plot(fallout, sensitivity, color = 'darkorange')
```



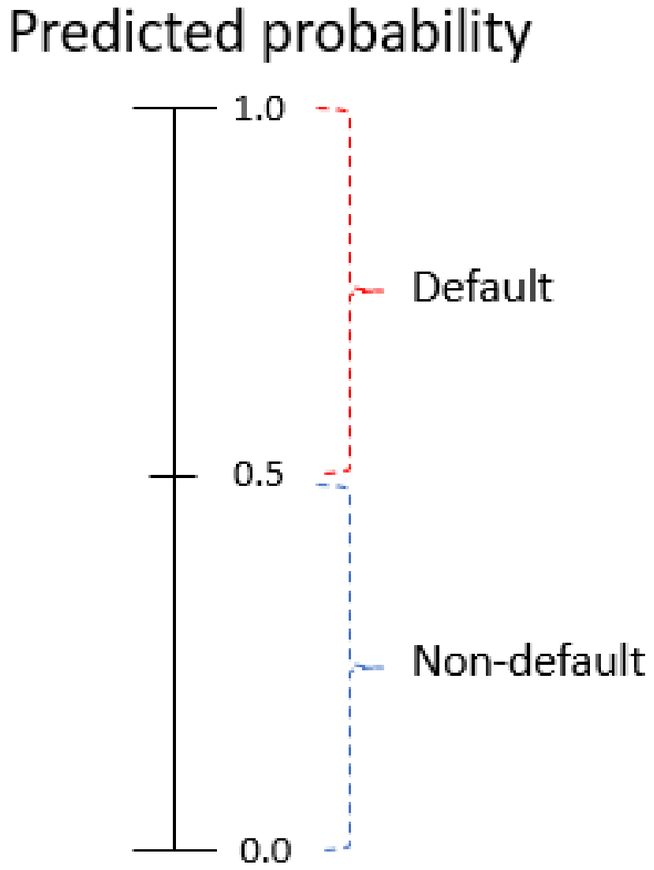
Analyzing ROC charts

- Area Under Curve (AUC): area between curve and random prediction



Default thresholds

- Threshold: at what point a probability is a default



Predicted probability	New loan_status
0.45	0 (non-default
0.87	1 (default)

Setting the threshold

- Relabel loans based on our threshold of `0.5`

```
preds = clf_logistic.predict_proba(X_test)
preds_df = pd.DataFrame(preds[:,1], columns = ['prob_default'])
preds_df['loan_status'] = preds_df['prob_default'].apply(lambda x: 1 if x > 0.5 else 0)
```

	prob_default	loan_status
11779	0.079626	0
11780	0.051979	0
11781	0.522450	1
11782	0.370478	0
11783	0.123786	0

Credit classification reports

- `classification_report()` within scikit-learn

```
from sklearn.metrics import classification_report
classification_report(y_test, preds_df['loan_status'], target_names=target_names)
```

	precision	recall	f1-score	support
Non-Default	0.79	0.99	0.88	9198
Default	0.67	0.04	0.07	2586
micro avg	0.78	0.78	0.78	11784
macro avg	0.73	0.52	0.47	11784
weighted avg	0.76	0.78	0.70	11784

Selecting classification metrics

- Select and store specific components from the `classification_report()`
- Use the `precision_recall_fscore_support()` function from scikit-learn

	precision	recall	f1-score	support
Non-Default	0.79	0.99	0.88	9198
Default	0.67	0.04	0.07	2586
micro avg	0.78	0.78	0.78	11784
macro avg	0.73	0.52	0.47	11784
weighted avg	0.76	0.78	0.70	11784

```
from sklearn.metrics import precision_recall_fscore_support
precision_recall_fscore_support(y_test,preds_df['loan_status'])[1][1]
```

Let's practice!

CREDIT RISK MODELING IN PYTHON

Model discrimination and impact

CREDIT RISK MODELING IN PYTHON

Michael Crabtree
Data Scientist, Ford Motor Company

datacamp

Confusion matrices

- Shows the number of correct and incorrect predictions for each `loan_status`

True Negatives (Predicted = 0, Actual = 0)	**False Positives** (Predicted = 1, Actual = 0)
False Negatives (Predicted = 0, Actual = 1)	**True Positives** (Predicted = 1, Actual = 1)

$$Precision(0) = \frac{TN}{TN + FN}$$

$$Precision(1) = \frac{TP}{TP + FP}$$

$$Recall(0) = \frac{TN}{TN + FP}$$

$$Recall(1) = \frac{TP}{TP + FN}$$

Default recall for loan status

- Default recall (or sensitivity) is the proportion of true defaults predicted

	precision	recall	f1-score	support
Non-Default	0.79	0.99	0.88	9198
Default	0.67	0.04	0.07	2586
micro avg	0.78	0.78	0.78	11784
macro avg	0.73	0.52	0.47	11784
weighted avg	0.76	0.78	0.70	11784

$$Recall(1) = \frac{TP}{TP + FN}$$

Recall portfolio impact

- Classification report - Underperforming Logistic Regression model

	precision	recall	f1-score	support
Non-Default	0.79	0.99	0.88	9198
Default	0.67	0.04	0.07	2586
micro avg	0.78	0.78	0.78	11784
macro avg	0.73	0.52	0.47	11784
weighted avg	0.76	0.78	0.70	11784

Recall portfolio impact

- Classification report - Underperforming Logistic Regression model

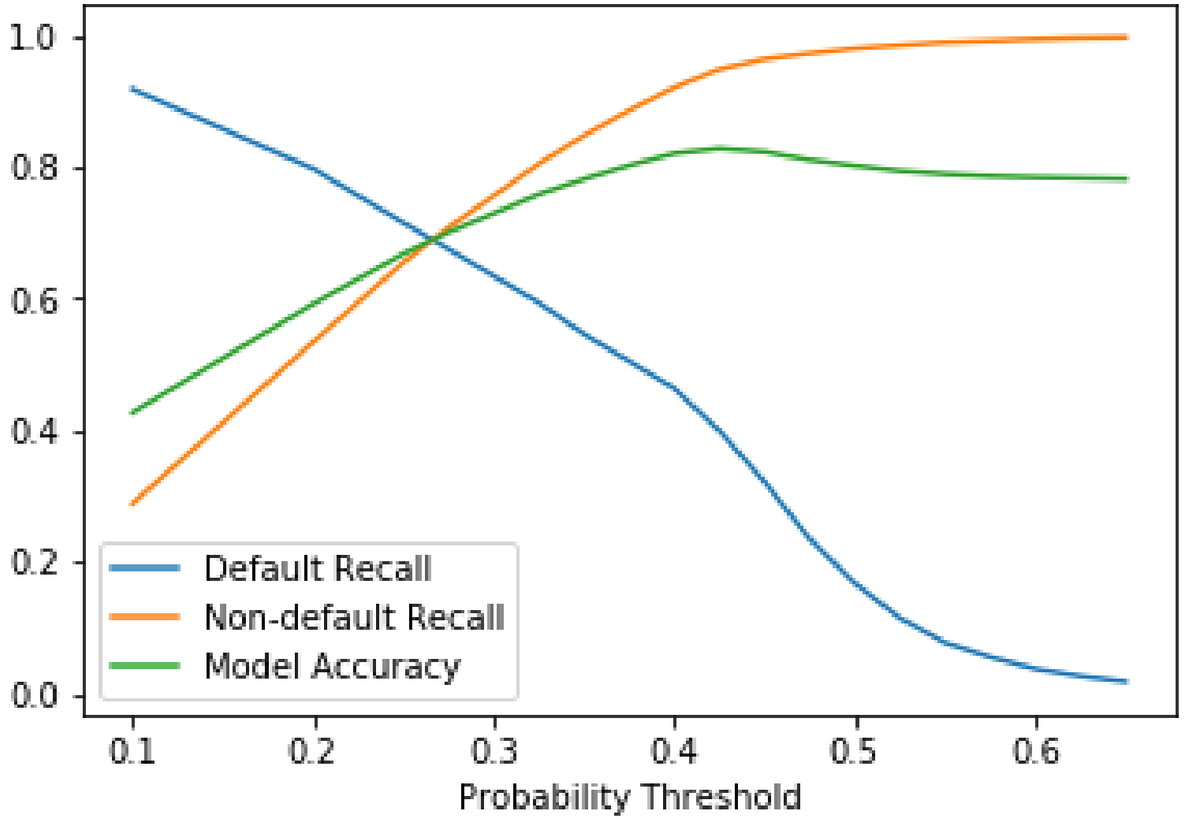
	precision	recall	f1-score	support
Non-Default	0.79	0.99	0.88	9198
Default	0.67	0.04	0.07	2586
micro avg	0.78	0.78	0.78	11784
macro avg	0.73	0.52	0.47	11784
weighted avg	0.76	0.78	0.70	11784

- Number of true defaults: 50,000

Loan Amount	Defaults Predicted / Not Predicted	Estimated Loss on Defaults
$50	.04 / .96	(50000 x .96) x 50 = $2,400,000

Recall, precision, and accuracy

- Difficult to maximize all of them because there is a trade-off



Let's practice!

CREDIT RISK MODELING IN PYTHON

Gradient boosted trees with XGBoost

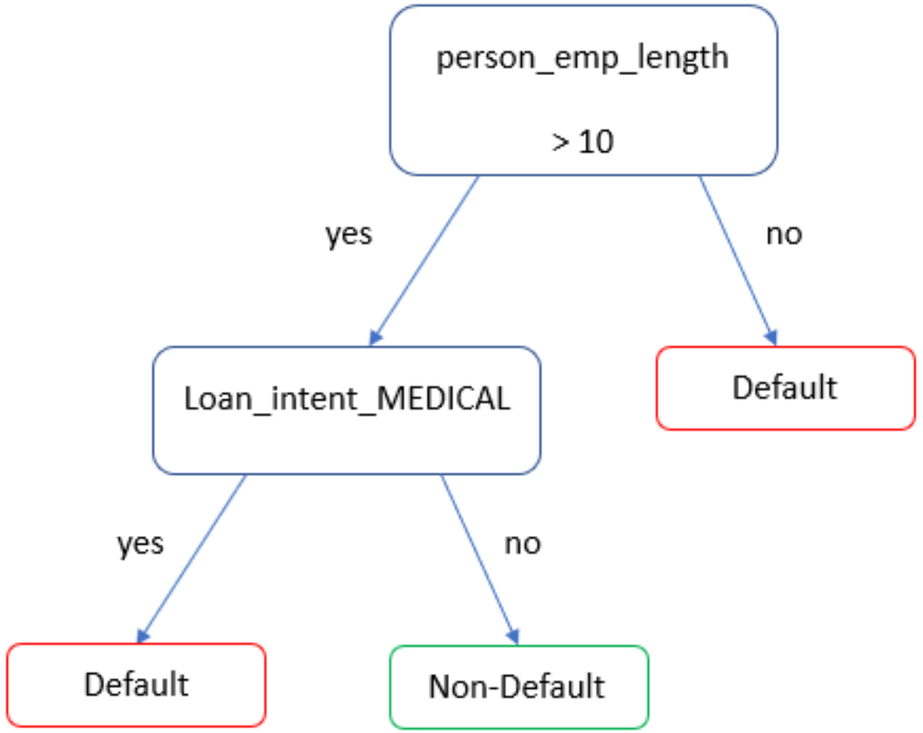
CREDIT RISK MODELING IN PYTHON

Michael Crabtree
Data Scientist, Ford Motor Company

datacamp

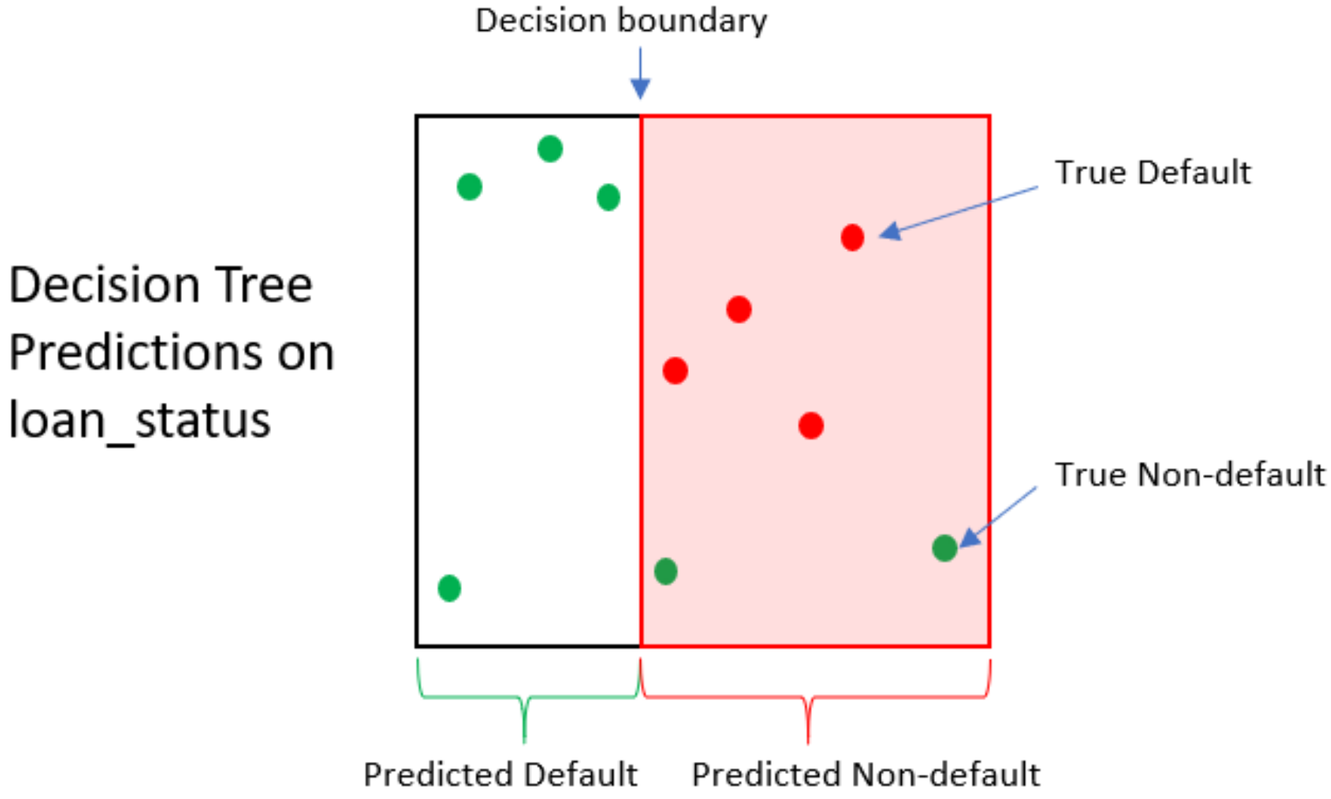
Decision trees

- Creates predictions similar to logistic regression
- Not structured like a regression



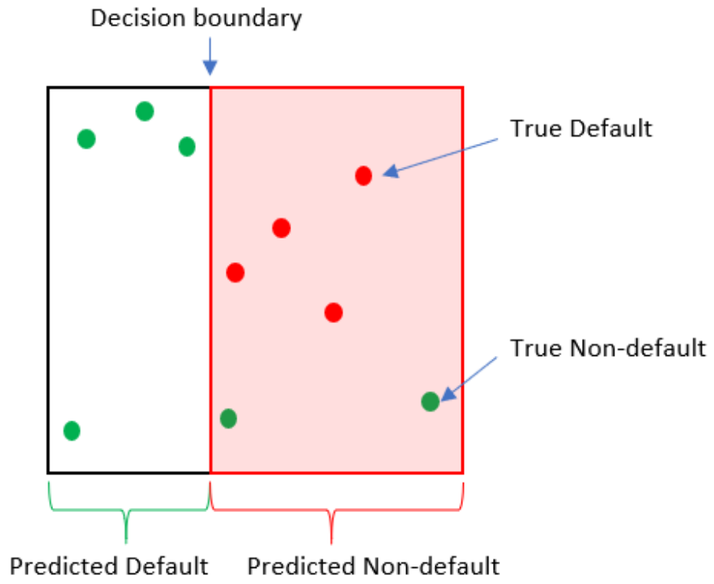
Decision trees for loan status

- Simple decision tree for predicting `loan_status` probability of default



Decision tree impact

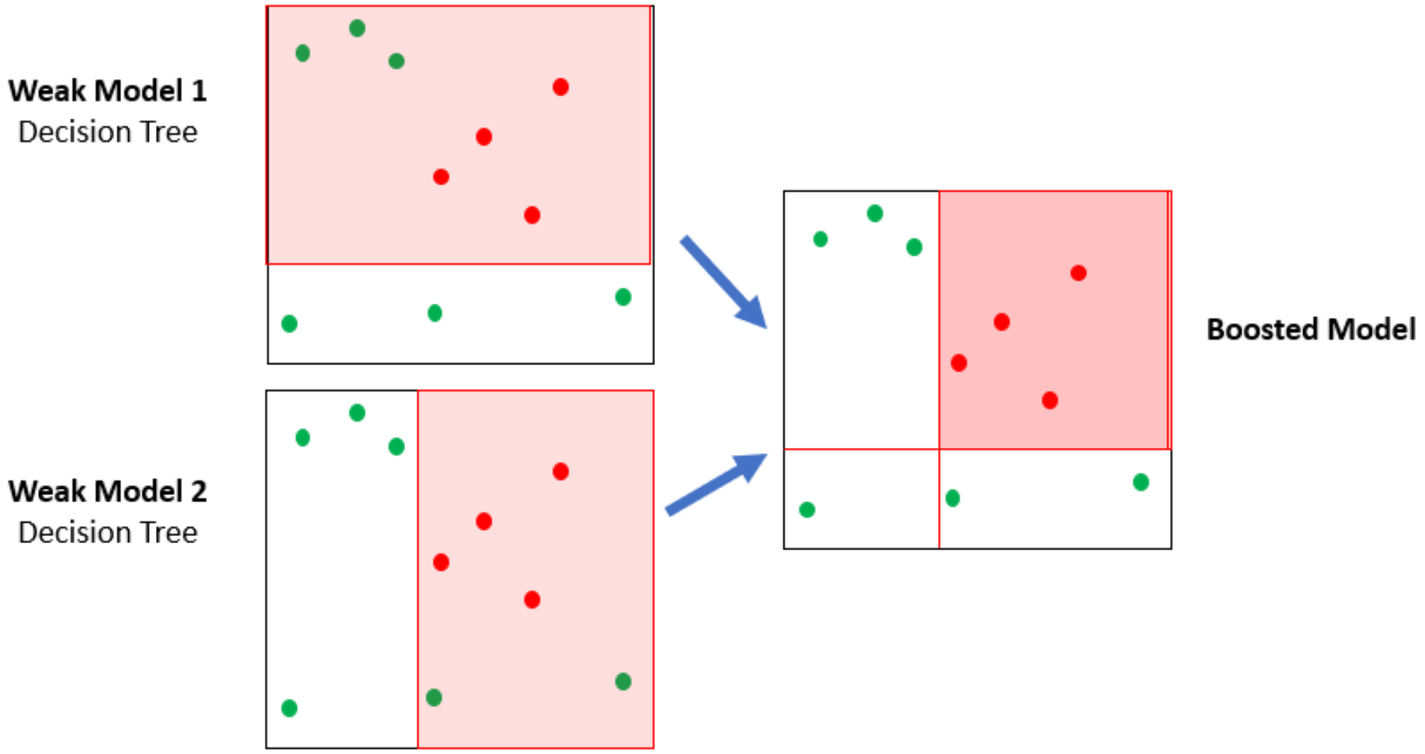
Decision Tree Predictions on loan_status



Loan	True loan status	Pred. Loan Status	Loan payoff value	Selling Value	Gain/Loss
1	0	1	$1,500	$250	-$1,250
2	0	1	$1,200	$250	-$950

A forest of trees

- XGBoost uses many simplistic trees (ensemble)
- Each tree will be slightly better than a coin toss



Creating and training trees

- Part of the `xgboost` Python package, called `xgb` here
- Trains with `.fit()` just like the logistic regression model

```
# Create a logistic regression model
clf_logistic = LogisticRegression()
# Train the logistic regression
clf_logistic.fit(X_train, np.ravel(y_train))
```

```
# Create a gradient boosted tree model
clf_gbt = xgb.XGBClassifier()
# Train the gradient boosted tree
clf_gbt.fit(X_train,np.ravel(y_train))
```

Default predictions with XGBoost

- Predicts with both `.predict()` and `.predict_proba()`
  - `.predict_proba()` produces a value between `0` and `1`
  - `.predict()` produces a `1` or `0` for `loan_status`

```
# Predict probabilities of default
gbt_preds_prob = clf_gbt.predict_proba(X_test)
# Predict loan_status as a 1 or 0
gbt_preds = clf_gbt.predict(X_test)
```

```
# gbt_preds_prob
array([[0.059, 0.940], [0.121, 0.989]])
# gbt_preds
array([1, 1, 0...])
```

Hyperparameters of gradient boosted trees

- Hyperparameters: model parameters (settings) that cannot be learned from data
- Some common hyperparameters for gradient boosted trees
  - `learning_rate` : smaller values make each step more conservative
  - `max_depth` : sets how deep each tree can go, larger means more complex

```
xgb.XGBClassifier(learning_rate = 0.2,
                  max_depth = 4)
```

Let's practice!

CREDIT RISK MODELING IN PYTHON

Column selection for credit risk

CREDIT RISK MODELING IN PYTHON

Michael Crabtree
Data Scientist, Ford Motor Company

Choosing specific columns

- We've been using all columns for predictions

```
# Selects a few specific columns
X_multi = cr_loan_prep[['loan_int_rate','person_emp_length']]
```

```
# Selects all data except loan_status
X = cr_loan_prep.drop('loan_status', axis = 1)
```

- How you can tell how important each column is
  - Logistic Regression: column coefficients
  - Gradient Boosted Trees: ?

Column importances

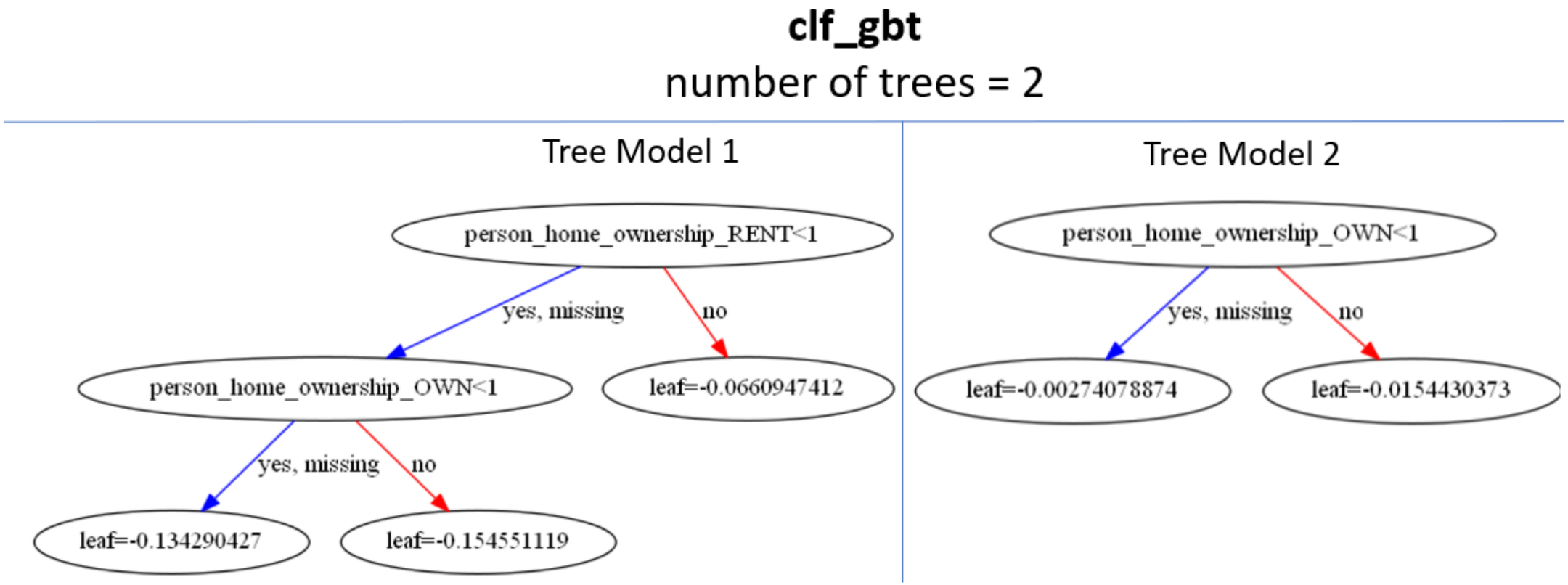
- Use the `.get_booster()` and `.get_score()` methods
  - Weight: the number of times the column appears in all trees

```
# Train the model
clf_gbt.fit(X_train,np.ravel(y_train))
# Print the feature importances
clf_gbt.get_booster().get_score(importance_type = 'weight')
```

```
{'person_home_ownership_RENT': 1, 'person_home_ownership_OWN': 2}
```

Column importance interpretation

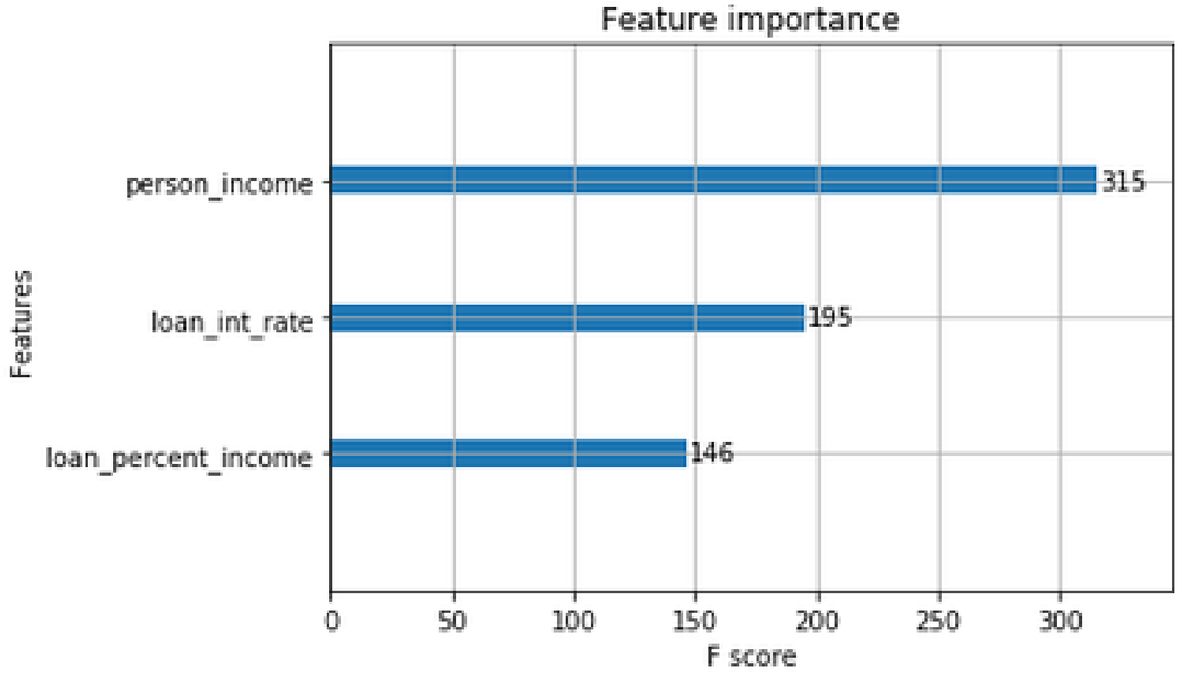
```
# Column importances from importance_type = 'weight'
{'person_home_ownership_RENT': 1, 'person_home_ownership_OWN': 2}
```



Plotting column importances

- Use the `plot_importance()` function

```
xgb.plot_importance(clf_gbt, importance_type = 'weight')
{'person_income': 315, 'loan_int_rate': 195, 'loan_percent_income': 146}
```



Choosing training columns

- Column importance is used to sometimes decide which columns to use for training
- Different sets affect the performance of the models

Columns	Importances	Model Accuracy	Model Default Recall
loan_int_rate, person_emp_length	(100, 100)	0.81	0.67
loan_int_rate, person_emp_length, loan_percent_income	(98, 70, 5)	0.84	0.52

F1 scoring for models

- Thinking about accuracy and recall for different column groups is time consuming
- F1 score is a single metric used to look at both accuracy and recall

$$F1\ Score = 2 * (\frac{precision * recall}{precision + recall})$$

- Shows up as a part of the `classification_report()`

	precision	recall	f1-score	support
Non-Default	0.93	0.99	0.96	9198
Default	0.96	0.72	0.82	2586
micro avg	0.93	0.93	0.93	11784
macro avg	0.94	0.85	0.89	11784
weighted avg	0.93	0.93	0.93	11784

Let's practice!

CREDIT RISK MODELING IN PYTHON

Cross validation for credit models

CREDIT RISK MODELING IN PYTHON

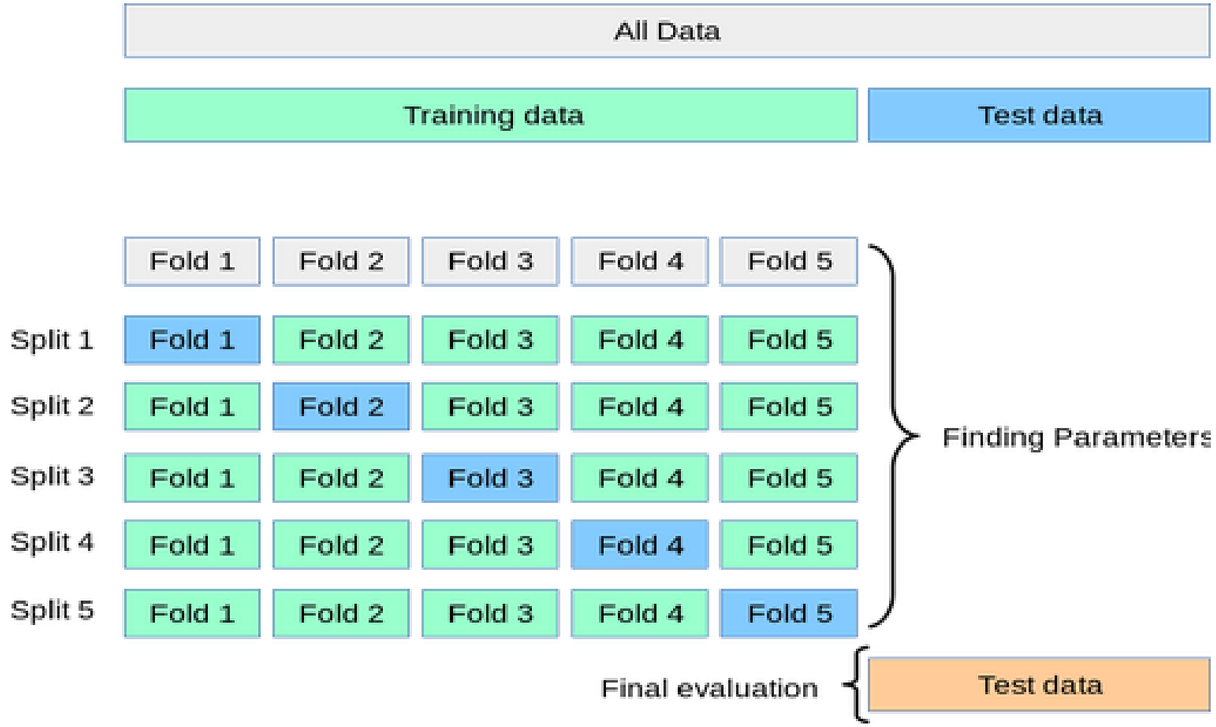
Michael Crabtree
Data Scientist, Ford Motor Company

Cross validation basics

- Used to train and test the model in a way that simulates using the model on new data
- Segments training data into different pieces to estimate future performance
- Uses `DMatrix`, an internal structure optimized for `XGBoost`
- Early stopping tells cross validation to stop after a scoring metric has not improved after a number of iterations

How cross validation works

- Processes parts of training data as (called folds) and tests against unused part
- Final testing against the actual test set



[1] https://scikit [2] learn.org/stable/modules/cross_validation.html

Setting up cross validation within XGBoost

```
# Set the number of folds
n_folds = 2
# Set early stopping number
early_stop = 5
# Set any specific parameters for cross validation
params = {'objective': 'binary:logistic',
          'seed': 99, 'eval_metric':'auc'}
```

- `'binary':'logistic'` is used to specify classification for `loan_status`
- `'eval_metric':'auc'` tells XGBoost to score the model's performance on AUC

Using cross validation within XGBoost

```
# Restructure the train data for xgboost
DTrain = xgb.DMatrix(X_train, label = y_train)
# Perform cross validation
xgb.cv(params, DTrain, num_boost_round = 5, nfold=n_folds,
       early_stopping_rounds=early_stop)
```

- `DMatrix()` creates a special object for `xgboost` optimized for training

The results of cross validation

- Creates a data frame of the values from the cross validation

	train-auc-mean	train-auc-std	test-auc-mean	test-auc-std
0	0.898444	0.002041	0.892701	0.006615
1	0.907534	0.001368	0.899609	0.008587
2	0.914467	0.002170	0.908039	0.007474
3	0.919102	0.000843	0.911437	0.007616
4	0.923488	0.001320	0.914825	0.006873

Cross validation scoring

- Uses cross validation and scoring metrics with `cross_val_score()` function in scikit-learn

```
# Import the module
from sklearn.model_selection import cross_val_score
# Create a gbt model
xg = xgb.XGBClassifier(learning_rate = 0.4, max_depth = 10)
# Use cross valudation and accuracy scores 5 consecutive times
cross_val_score(gbt, X_train, y_train, cv = 5)
```

```
array([0.92748092, 0.92575308, 0.93975392, 0.93378608, 0.93336163])
```

Let's practice!

CREDIT RISK MODELING IN PYTHON

Class imbalance in loan data

CREDIT RISK MODELING IN PYTHON

Michael Crabtree
Data Scientist, Ford Motor Company

Not enough defaults in the data

- The values of `loan_status` are the classes
  - Non-default: `0`
  - Default: `1`

```
y_train['loan_status'].value_counts()
```

loan_status	Training Data Count	Percentage of Total
0	13,798	78%
1	3,877	22%

Model loss function

- Gradient Boosted Trees in `xgboost` use a loss function of log-loss
  - The goal is to minimize this value

$$Log\ Loss = -\frac{1}{N}\sum_{i=1}^{N}[y_i * \log(p_i) + (1 - y_i) * \log(1 - p_i)]$$

True loan status	Predicted probability	Log Loss
1	0.1	2.3
0	0.9	2.3

- An inaccurately predicted default has more negative financial impact

The cost of imbalance

- A false negative (default predicted as non-default) is much more costly

Person	Loan Amount	Potential Profit	Predicted Status	Actual Status	Losses
A	$1,000	$10	Default	Non-Default	-$10
B	$1,000	$10	Non-Default	Default	-$1,000

- Log-loss for the model is the same for both, our actual losses is not

Causes of imbalance

- Data problems
  - Credit data was not sampled correctly
  - Data storage problems
- Business processes:
  - Measures already in place to not accept probable defaults
  - Probable defaults are quickly sold to other firms
- Behavioral factors:
  - Normally, people do not default on their loans
    - The less often they default, the higher their credit rating

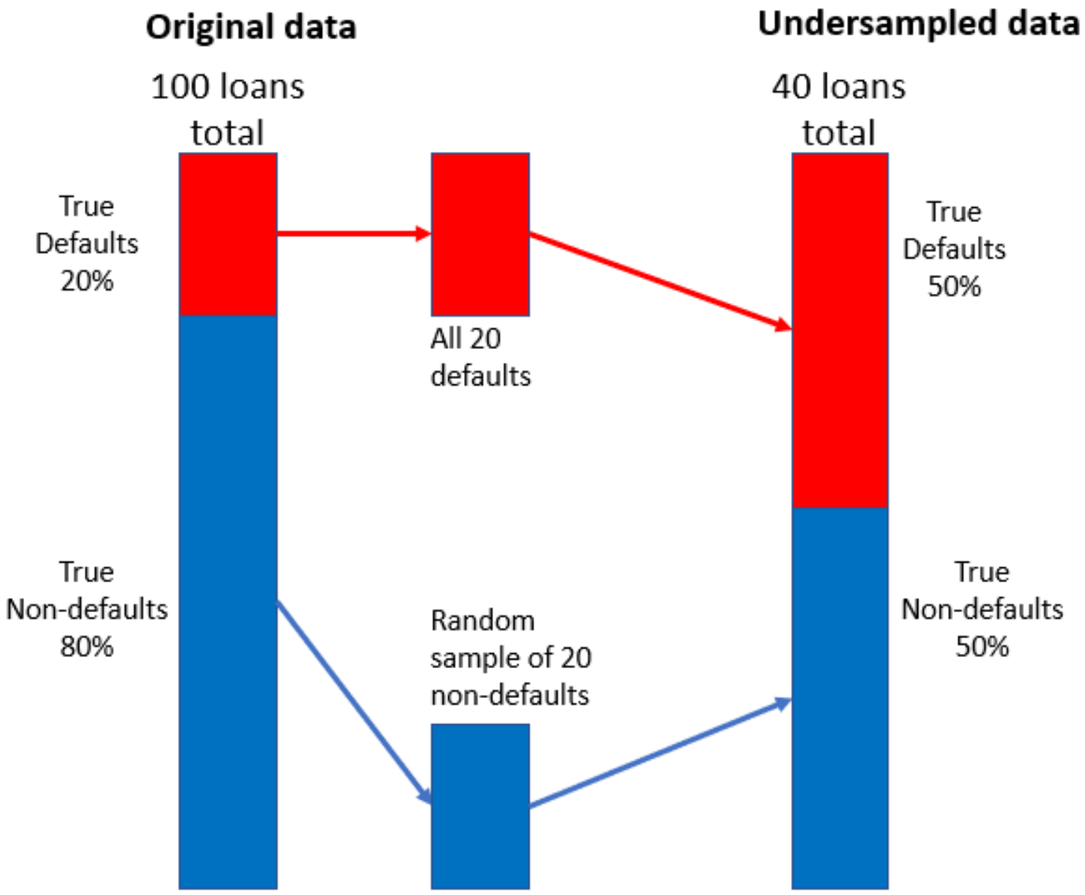
Dealing with class imbalance

- Several ways to deal with class imbalance in data

Method	Pros	Cons
Gather more data	Increases number of defaults	Percentage of defaults may not change
Penalize models	Increases recall for defaults	Model requires more tuning and maintenance
Sample data differently	Least technical adjustment	Fewer defaults in data

Undersampling strategy

- Combine smaller random sample of non-defaults with defaults



Combining the split data sets

- Test and training set must be put back together
- Create two new sets based on actual `loan_status`

```
# Concat the training sets
X_y_train = pd.concat([X_train.reset_index(drop = True),
                       y_train.reset_index(drop = True)], axis = 1)
# Get the counts of defaults and non-defaults
count_nondefault, count_default = X_y_train['loan_status'].value_counts()
# Separate nondefaults and defaults
nondefaults = X_y_train[X_y_train['loan_status'] == 0]
defaults = X_y_train[X_y_train['loan_status'] == 1]
```

Undersampling the non-defaults

- Randomly sample data set of non-defaults
- Concatenate with data set of defaults

```
# Undersample the non-defaults using sample() in pandas
nondefaults_under = nondefaults.sample(count_default)
# Concat the undersampled non-defaults with the defaults
X_y_train_under = pd.concat([nondefaults_under.reset_index(drop = True),
                             defaults.reset_index(drop = True)], axis=0)
```

Let's practice!

CREDIT RISK MODELING IN PYTHON

Model evaluation and implementation

CREDIT RISK MODELING IN PYTHON

Michael Crabtree
Data Scientist, Ford Motor Company

datacamp

Comparing classification reports

- Create the reports with `classification_report()` and compare

Logistic Regression

	precision	recall	f1-score	support
Non-Default	0.81	0.98	0.89	9198
Default	0.71	0.17	0.27	2586
micro avg	0.80	0.80	0.80	11784
macro avg	0.76	0.57	0.58	11784
weighted avg	0.79	0.80	0.75	11784

Gradient Boosted Tree

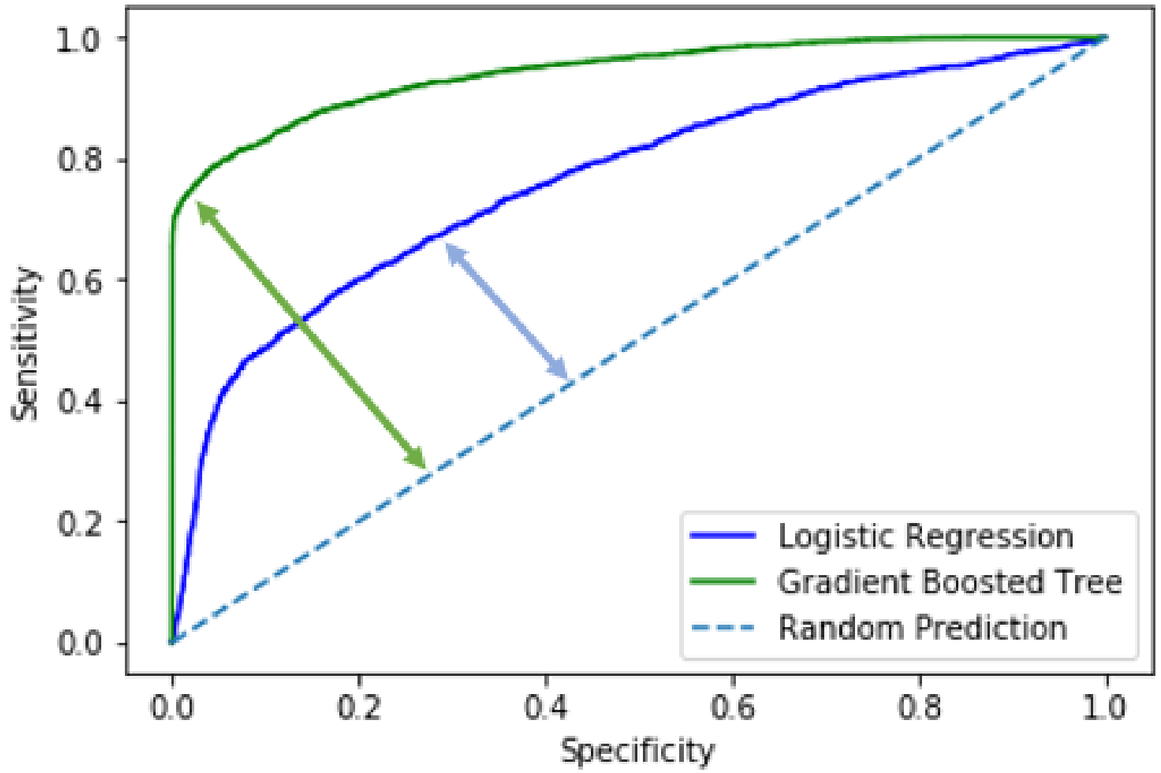
	precision	recall	f1-score	support
Non-Default	0.90	0.98	0.94	9198
Default	0.91	0.63	0.74	2586
micro avg	0.90	0.90	0.90	11784
macro avg	0.91	0.80	0.84	11784
weighted avg	0.90	0.90	0.90	11784

$$F_1Score = 2 * \left(\frac{precision * recall}{precision + recall}\right)$$

$$Macro\ Average = \frac{F_1Score(Default) + F_1Score(NonDefault)}{2}$$

ROC and AUC analysis

- Models with better performance will have more lift
- More lift means the AUC score is higher



Model calibration

- We want our probabilities of default to accurately represent the model's confidence level
  - The probability of default has a degree of uncertainty in it's predictions
- A sample of loans and their predicted probabilities of default should be close to the percentage of defaults in that sample

Sample of loans	Average predicted PD	Sample percentage of actual defaults	Calibrated?
10	0.12	0.12	Yes
10	0.25	0.65	No

1 http://datascienceassn.org/sites/default/files/Predicting%20good%20probabilities%20with%20supervised%20le

Calculating calibration

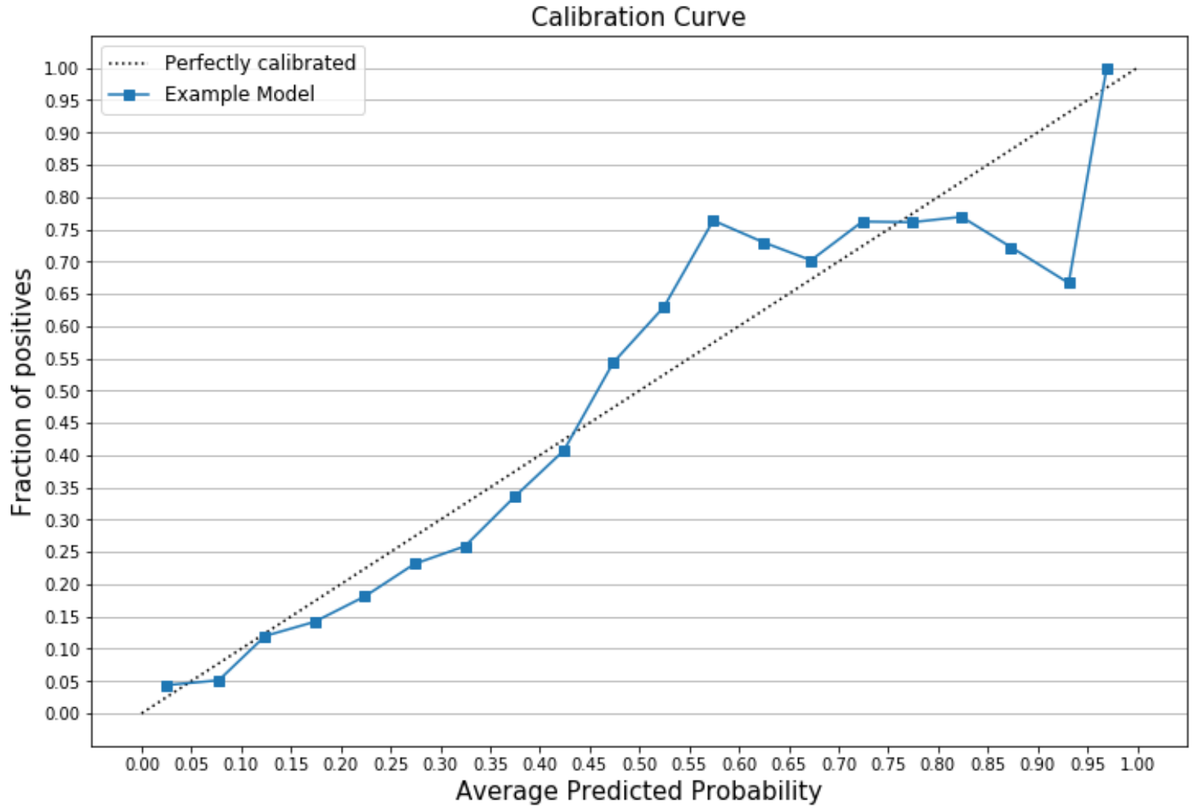
- Shows percentage of true defaults for each predicted probability
- Essentially a line plot of the results of `calibration_curve()`

```
from sklearn.calibration import calibration_curve
calibration_curve(y_test, probabilities_of_default, n_bins = 5)
```

```
# Fraction of positives
(array([0.09602649, 0.19521012, 0.62035996, 0.67361111]),
# Average probability
 array([0.09543535, 0.29196742, 0.46898465, 0.65512207]))
```

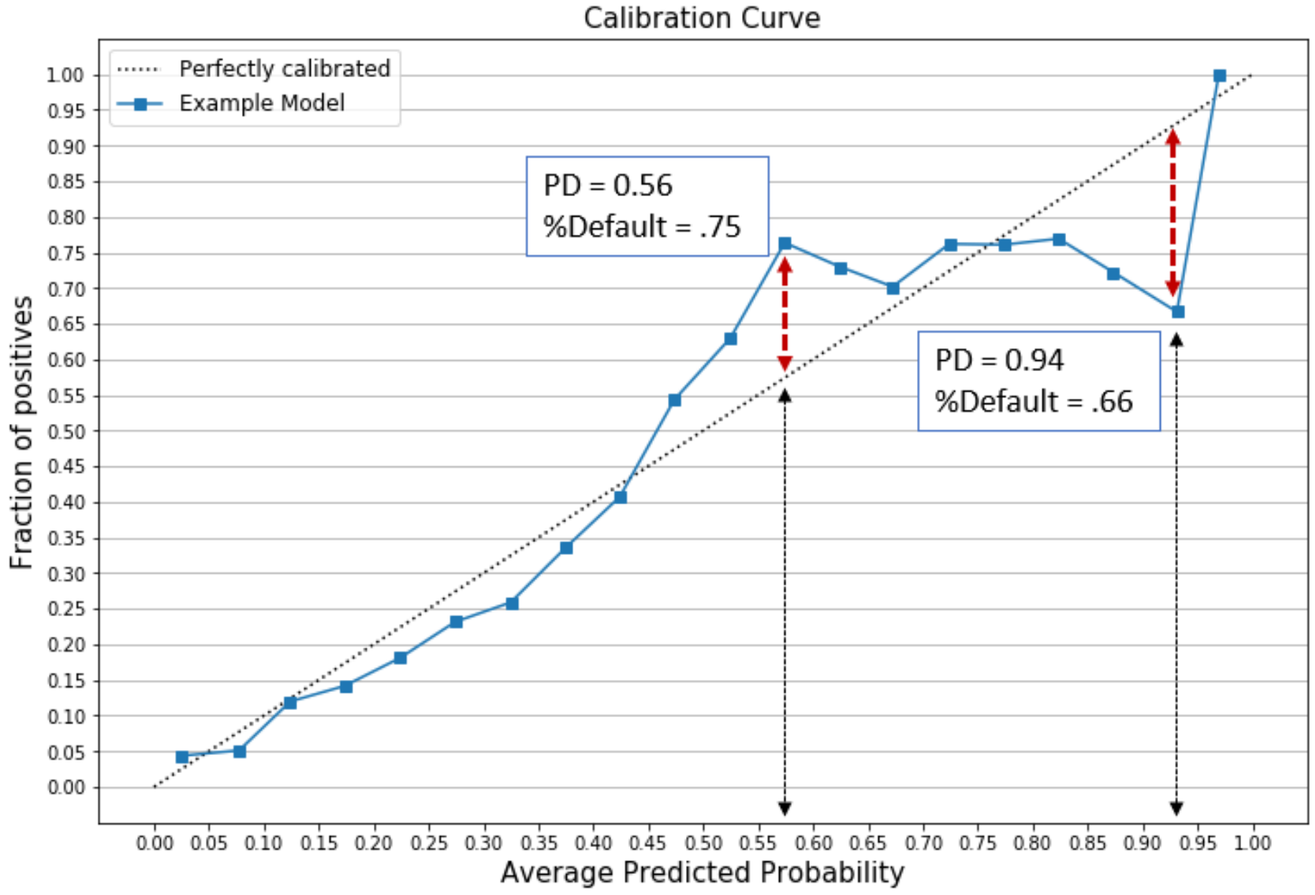
Plotting calibration curves

```
plt.plot(mean_predicted_value, fraction_of_positives, label="%s" % "Example Model")
```

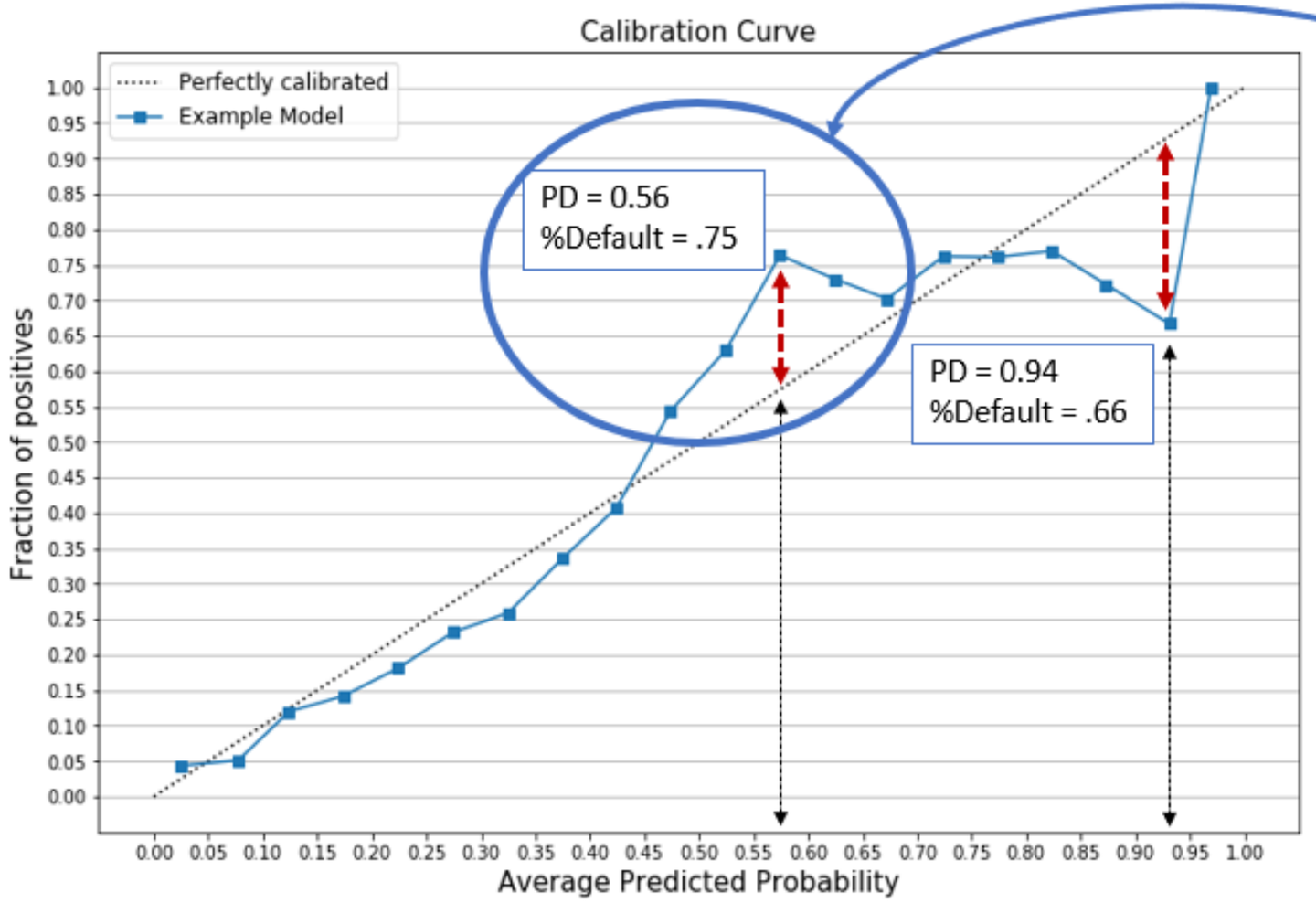


Checking calibration curves

- As an example, two events selected (above and below perfect line)



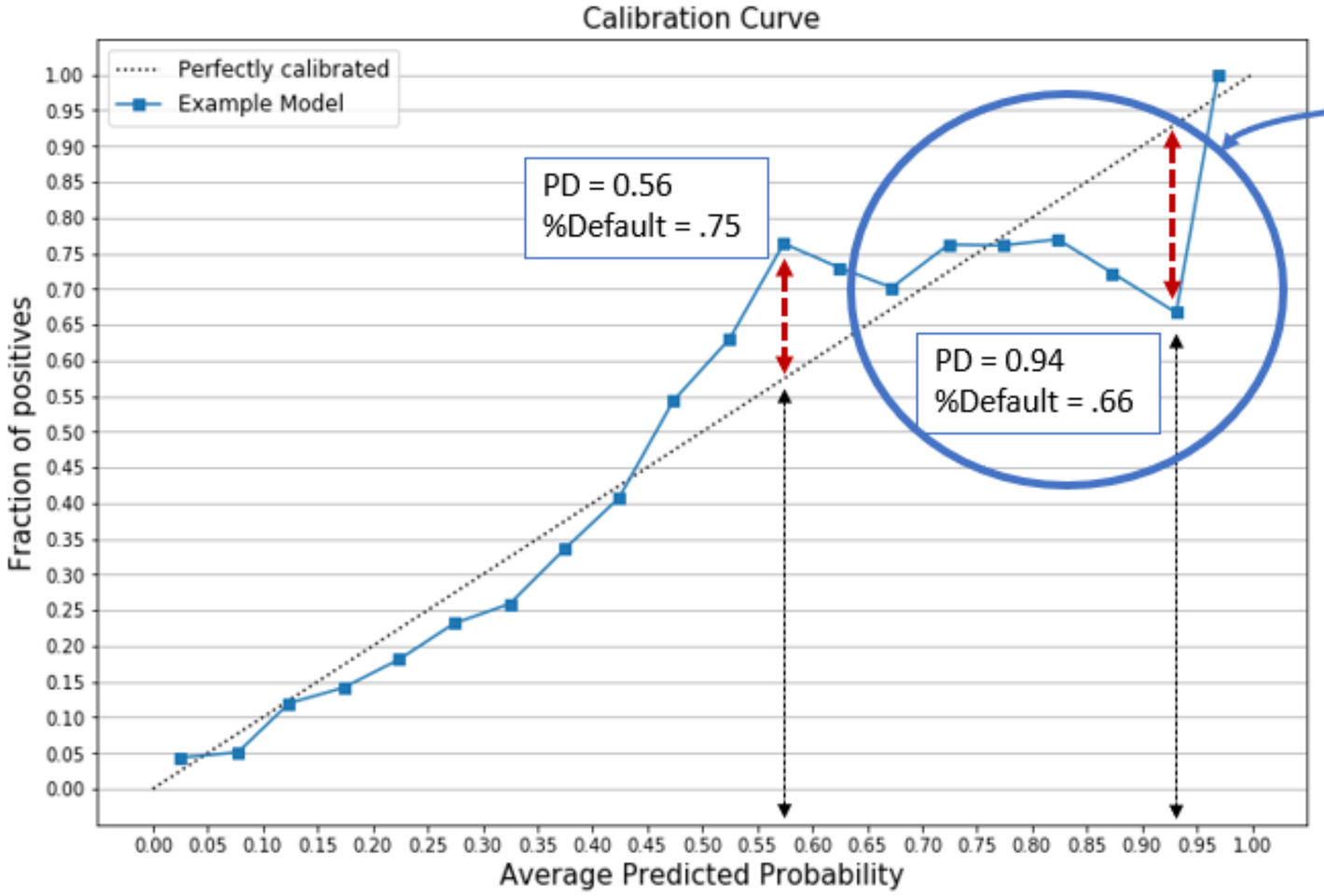
Calibration curve interpretation



When model predictions are around 0.56, more of the loans are actually defaults

This is where those costly false negatives occur, and a default could result in a loss for the entire loan amount

Calibration curve interpretation



When model predictions are around 0.94, more of the loans are actually non-defaults

This is where the majority of the false positives exist. These are loans we might deny because our model thinks they are defaults. This is a loss of profit for each loan here

Let's practice!

CREDIT RISK MODELING IN PYTHON

Credit acceptance rates

CREDIT RISK MODELING IN PYTHON

Michael Crabtree
Data Scientist, Ford Motor Company

Thresholds and loan status

- Previously we set a threshold for a range of `prob_default` values
  - This was used to change the predicted `loan_status` of the loan

```
preds_df['loan_status'] = preds_df['prob_default'].apply(lambda x: 1 if x > 0.4 else 0)
```

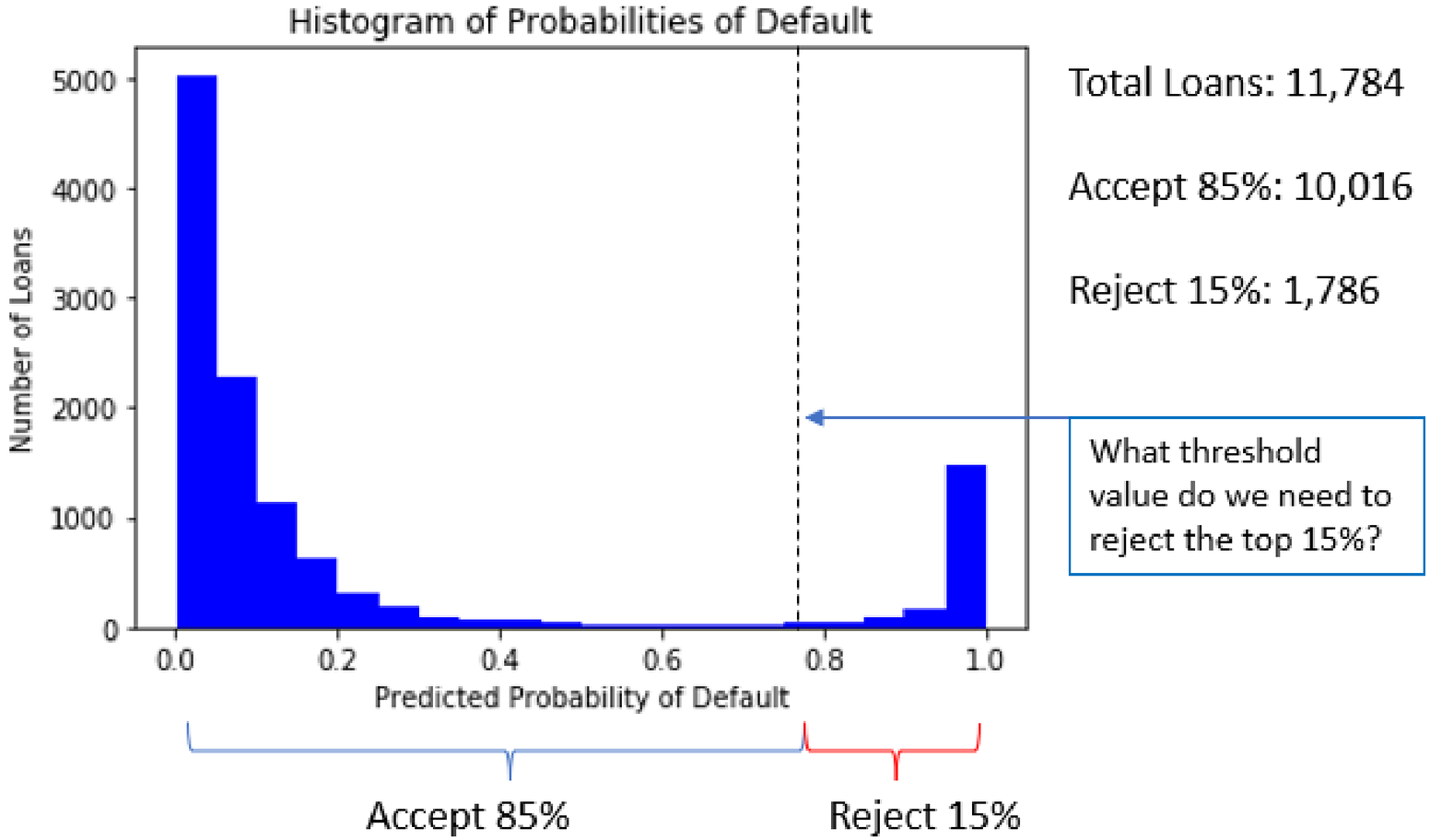
Loan	prob_default	threshold	loan_status
1	0.25	0.4	0
2	0.42	0.4	1
3	0.75	0.4	1

Thresholds and acceptance rate

- Use model predictions to set better thresholds
  - Can also be used to approve or deny new loans
- For all new loans, we want to deny probable defaults
  - Use the test data as an example of new loans
- Acceptance rate: what percentage of new loans are accepted to keep the number of defaults in a portfolio low
  - Accepted loans which are defaults have an impact similar to false negatives

Understanding acceptance rate

- Example: Accept 85% of loans with the lowest `prob_default`



Calculating the threshold

- Calculate the threshold value for an 85% acceptance rate

```
import numpy as np
# Compute the threshold for 85% acceptance rate
threshold = np.quantile(prob_default, 0.85)
```

```
0.804
```

Loan	prob_default	Threshold	Predicted loan_status	Accept or Reject
1	0.65	0.804	0	Accept
2	0.85	0.804	1	Reject

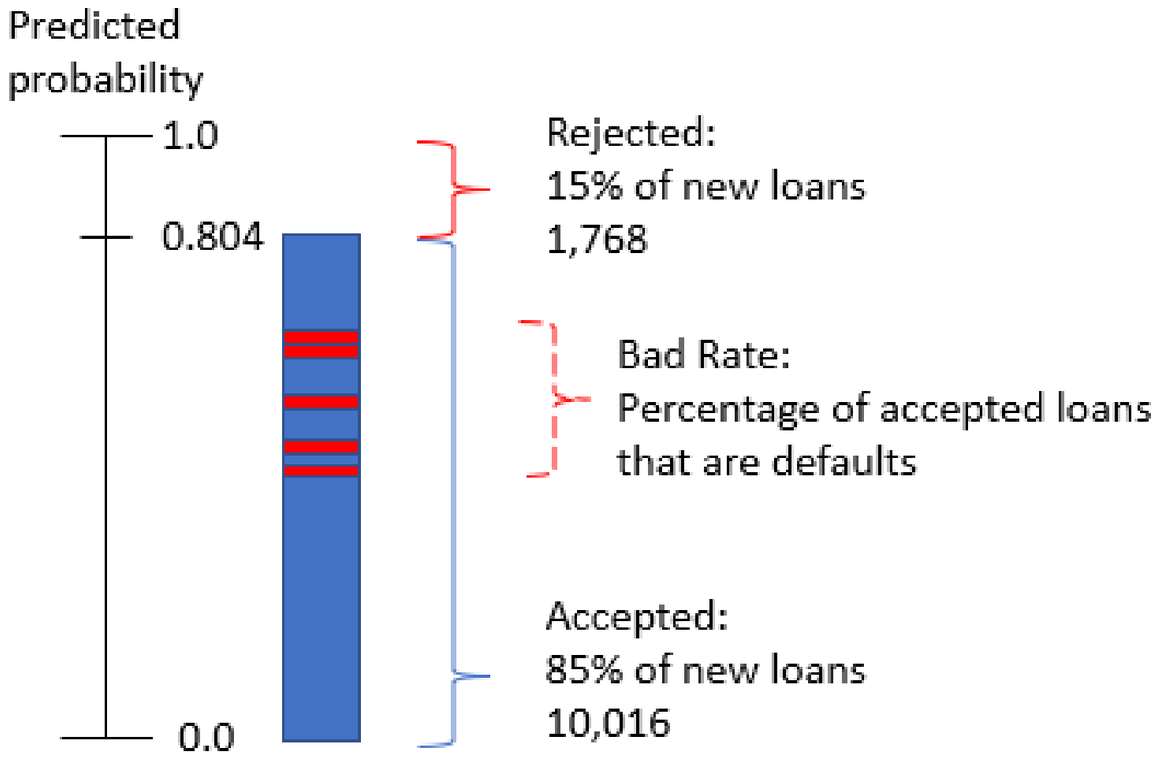
Implementing the calculated threshold

- Reassign `loan_status` values using the new threshold

```
# Compute the quantile on the probabilities of default
preds_df['loan_status'] = preds_df['prob_default'].apply(lambda x: 1 if x > 0.804 else 0)
```

Bad Rate

- Even with a calculated threshold, some of the accepted loans will be defaults
- These are loans with `prob_default` values around where our model is not well calibrated



Bad rate calculation

$$Bad\ Rate = \frac{Accepted\ Defaults}{Total\ Accepted\ Loans}$$

```
#Calculate the bad rate
np.sum(accepted_loans['true_loan_status']) / accepted_loans['true_loan_status'].count()
```

- If non-default is `0`, and default is `1` then the `sum()` is the count of defaults
- The `.count()` of a single column is the same as the row count for the data frame

Let's practice!

CREDIT RISK MODELING IN PYTHON

Credit strategy and minimum expected loss

CREDIT RISK MODELING IN PYTHON

Michael Crabtree
Data Scientist, Ford Motor Company

Selecting acceptance rates

- First acceptance rate was set to 85%, but other rates might be selected as well
- Two options to test different rates:
  - Calculate the threshold, bad rate, and losses manually
  - Automatically create a table of these values and select an acceptance rate
- The table of all the possible values is called a strategy table

Setting up the strategy table

- Set up arrays or lists to store each value

```
# Set all the acceptance rates to test
accept_rates = [1.0, 0.95, 0.9, 0.85, 0.8, 0.75, 0.7, 0.65, 0.6, 0.55,
                0.5, 0.45, 0.4, 0.35, 0.3, 0.25, 0.2, 0.15, 0.1, 0.05]
# Create lists to store thresholds and bad rates
thresholds = []
bad_rates = []
```

Calculating the table values

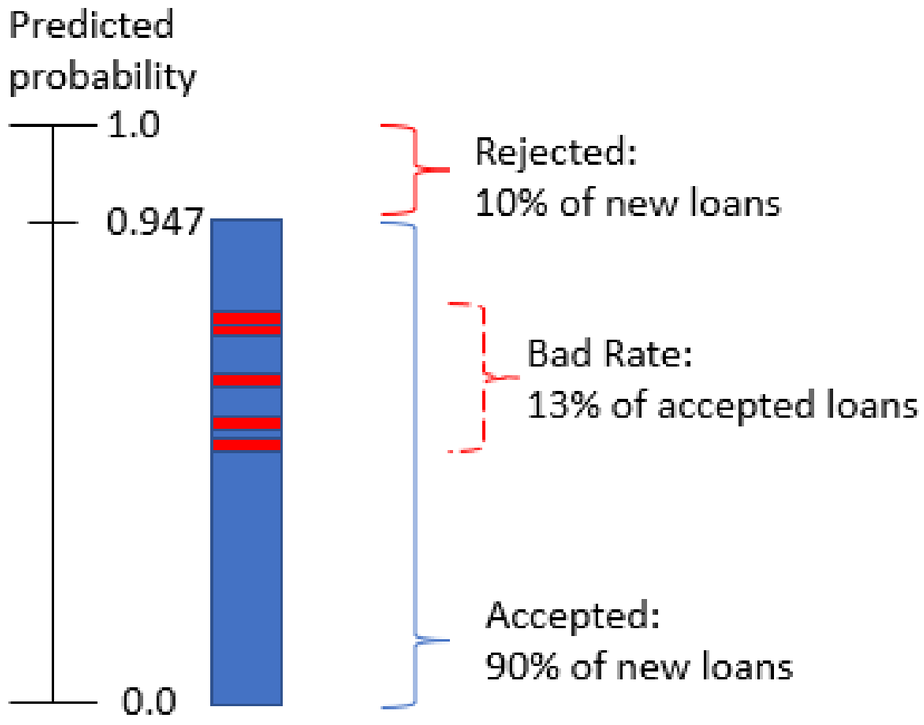
- Calculate the threshold and bad rate for all acceptance rates

```
for rate in accept_rates:
    # Calculate threshold
    threshold = np.quantile(preds_df['prob_default'], rate).round(3)
    # Store threshold value in a list
    thresholds.append(np.quantile(preds_gbt['prob_default'], rate).round(3))
    # Apply the threshold to reassign loan_status
    test_pred_df['pred_loan_status'] = \
        test_pred_df['prob_default'].apply(lambda x: 1 if x > thresh else 0)
    # Create accepted loans set of predicted non-defaults
    accepted_loans = test_pred_df[test_pred_df['pred_loan_status'] == 0]
    # Calculate and store bad rate
    bad_rates.append(np.sum((accepted_loans['true_loan_status'])
             / accepted_loans['true_loan_status'].count()).round(3))
```

Strategy table interpretation

```
strat_df = pd.DataFrame(zip(accept_rates, thresholds, bad_rates),
                        columns = ['Acceptance Rate','Threshold','Bad Rate'])
```

Acceptance Rate	Threshold	Bad Rate
1.00	0.999	0.219
0.95	0.988	0.177
0.90	0.947	0.133
0.85	0.503	0.097
0.80	0.330	0.078
0.75	0.227	0.066
0.70	0.163	0.055



Adding accepted loans

- The number of loans accepted for each acceptance rate
  - Can use `len()` or `.count()`

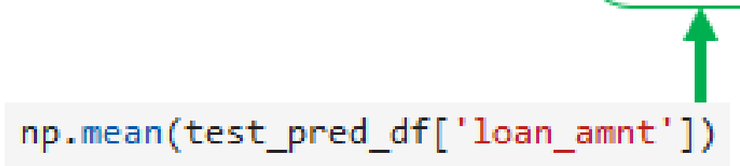
```
len(test_pred_df[test_pred_df['prob_default'] < np.quantile(test_pred_df['prob_default'], accept_rate)])
```

Acceptance Rate	Threshold	Bad Rate	Num Accepted Loans	Avg Loan Amnt	Estimated Value
1.00	0.999	0.219	11379	9556.28	61112391.49
0.95	0.988	0.177	10591	9556.28	65382022.72
0.90	0.947	0.133	10025	9556.28	70318452.94
0.85	0.503	0.097	9390	9556.28	72325176.18
0.80	0.330	0.078	8857	9556.28	71436136.33
0.75	0.227	0.066	8229	9556.28	68258329.21
0.70	0.163	0.055	7685	9556.28	65361610.50

Adding average loan amount

- Average `loan_amnt` from the test set data

Acceptance Rate	Threshold	Bad Rate	Num Accepted Loans	Avg Loan Amnt	Estimated Value
1.00	0.999	0.219	11379	9556.28	61112391.49
0.95	0.988	0.177	10591	9556.28	65382022.72
0.90	0.947	0.133	10025	9556.28	70318452.94
0.85	0.503	0.097	9390	9556.28	72325176.18
0.80	0.330	0.078	8857	9556.28	71436136.33
0.75	0.227	0.066	8229	9556.28	68258329.21
0.70	0.163	0.055	7685	9556.28	65361610.50



```
np.mean(test_pred_df['loan_amnt'])
```

Estimating portfolio value

- Average value of accepted loan non-defaults minus average value of accepted defaults
- Assumes each default is a loss of the `loan_amnt`

Acceptance Rate	Threshold	Bad Rate	Num Accepted Loans	Avg Loan Amnt	Estimated Value
1.00	0.999	0.219	11379	9556.28	61112391.49
0.95	0.988	0.177	10591	9556.28	65382022.72
0.90	0.947	0.133	10025	9556.28	70318452.94
0.85	0.503	0.097	9390	9556.28	72325176.18
0.80	0.330	0.078	8857	9556.28	71436136.33
0.75	0.227	0.066	8229	9556.28	68258329.21
0.70	0.163	0.055	7685	9556.28	65361610.50

```
((strat_df['Num Accepted Loans'] * (1 - strat_df['Bad Rate'])) * strat_df['Avg Loan Amnt'])
    - (strat_df['Num Accepted Loans'] * strat_df['Bad Rate'] * strat_df['Avg Loan Amnt'])
```

Total expected loss

- How much we expect to lose on the defaults in our portfolio

$$Total\ Expected\ Loss = \sum_{x=1}^{n} PD_x * LGD_x * EAD_x$$

```
# Probability of default (PD)
test_pred_df['prob_default']
# Exposure at default = loan amount (EAD)
test_pred_df['loan_amnt']
# Loss given default = 1.0 for total loss (LGD)
test_pred_df['loss_given_default']
```

Let's practice!

CREDIT RISK MODELING IN PYTHON

Course wrap up

CREDIT RISK MODELING IN PYTHON

Michael Crabtree
Data Scientist, Ford Motor Company

Your journey...so far

- Prepare credit data for machine learning models
  - Important to understand the data
  - Improving the data allows for high performing simple models
- Develop, score, and understand logistic regressions and gradient boosted trees
- Analyze the performance of models by changing the data
- Understand the financial impact of results
- Implement the model with an understanding of strategy

Risk modeling techniques

- The models and framework in this course:
  - Discrete-time hazard model (point in time): the probability of default is a point-in-time event
  - Stuctural model framework: the model explains the default even based on other factors
- Other techniques
  - Through-the-cycle model (continuous time): macro-economic conditions and other effects are used, but the risk is seen as an independent event
  - Reduced-form model framework: a statistical approach estimating probability of default as an independent Poisson-based event

Choosing models

- Many machine learning models available, but logistic regression and tree models were used
  - These models are simple and explainable
  - Their performance on probabilities is acceptable
- Many financial sectors prefer model interpretability
  - Complex or "black-box" models are a risk because the business cannot explain their decisions fully
  - Deep neural networks are often too complex

Tips from me to you

- Focus on the data
  - Gather as much data as possible
  - Use many different techniques to prepare and enhance the data
  - Learn about the business
  - Increase value through data
- Model complexity can be a two-edged sword
  - Really complex models may perform well, but are seen as a "black-box"
  - In many cases, business users will not accept a model they cannot understand
  - Complex models can be very large and difficult to put into production

Thank you!

CREDIT RISK MODELING IN PYTHON